

PROJECT OBSERVER

A Governed Multi-Agent Control Plane for Durable, Auditable AI Operations

SANITIZED TECHNICAL WHITE PAPER — REVISED EDITION

Purpose A deployment-oriented explanation beginning with the required hardware, then the governed agents, the external authoritative-state server, Model Context Protocol (MCP) connector boundary, Secure Shell (SSH) execution path, leases, handover, durable instructions, delegation, failover, and recovery.

Reference architecture. Deployment-specific names, addresses, credentials, identifiers, and filesystem paths are intentionally generalized.

Reader note: This paper expands specialized terms on first use and uses plain-language descriptions thereafter. In-System Mail means the durable internal messaging mechanism used by governed agents; it is not ordinary email. The Model Context Protocol (MCP) is the structured tool interface between the AI agent and external services. Secure Shell (SSH) is an encrypted transport, not a source of authority or truth.

1. 1. Required Hardware Setup

Project Observer does not require specialized AI hardware. The language model may run as a hosted service; the durable governance and execution state lives outside the conversation on infrastructure controlled by the operator. A minimal deployment therefore needs only the hardware required to host the external control plane and to reach the systems being managed.

Core principle The AI session is replaceable. The external governance host is the durable operational memory and authority source. Model Context Protocol (MCP) connectors and Secure Shell (SSH) are transport mechanisms used to reach that truth; they are not themselves the truth store.

1.1 Minimum physical components

Component	Minimum role in the architecture	Practical baseline
User workstation	Runs the browser/desktop client used to interact with the AI service. No authoritative Observer state is required to live here.	Modern workstation with reliable network access. No dedicated GPU required.
External governance server	Hosts the control plane, role/lease state, handovers, durable instruction records, connection directory, job state, evidence, brokers, and audit records.	Linux-capable x86-64 or ARM64 server/VM; 2-4 vCPU, 4-8 GB RAM, 40-80 GB SSD is sufficient for a small deployment.
Network path	Allows the MCP-facing integration to reach the governance server and allows the governance server to reach authorized managed targets.	Stable IP connectivity; encrypted transport; SSH/API access only where explicitly required.
Managed target systems	The repositories, servers, applications, databases, lab systems, or cloud APIs on which governed work is performed.	Existing infrastructure; no Observer-specific hardware requirement beyond reachable authorized interfaces.

1.2 Recommended production separation

For stronger assurance, separate the governance host from managed production systems. The governance host should be backed up and should not depend on any one chat session, connector process, or managed application remaining healthy. High-assurance deployments may place the audit sink or cryptographic signing material on a separately protected service so the same operating-system account that performs ordinary governance work cannot silently rewrite historical evidence.

Persistent storage for lease, handover, mail/instruction lifecycle, work-authority, job, and audit records.

Reliable clock synchronization because leases, expirations, signed messages, and checkpoints depend on time.

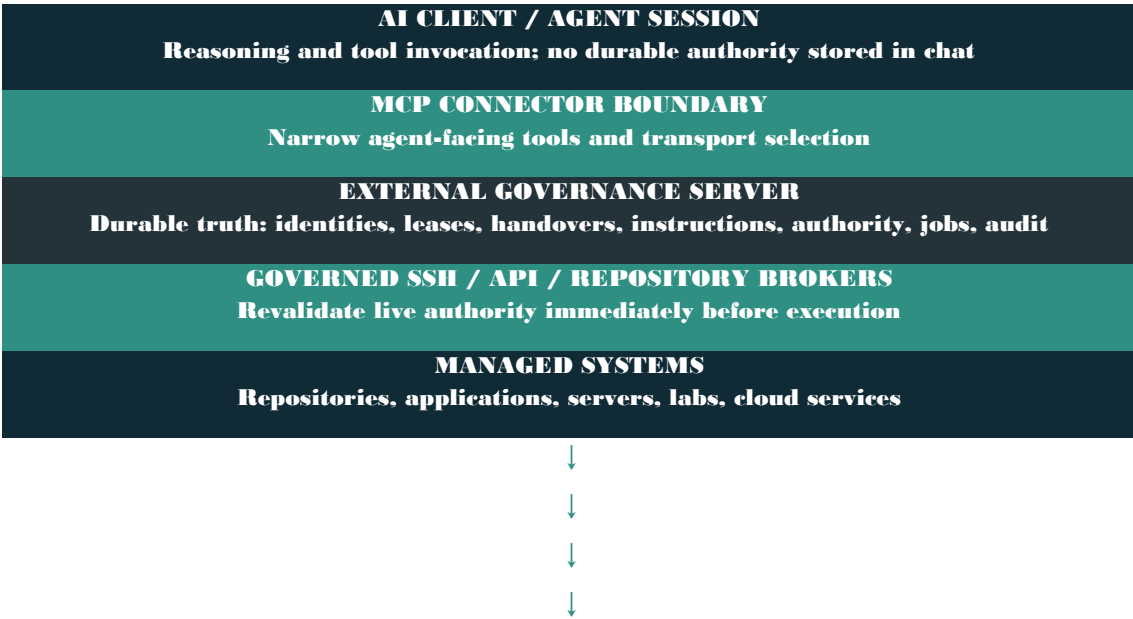
A backup/recovery path for the governance data store and configuration.

Restricted administrative access; routine agent work should use narrow brokers rather than unrestricted root shells.

Optional redundant connector lanes for availability, provided failover cannot broaden authority.

1.3 Network topology

Figure 1. Minimal physical / logical topology



A single small server can host the governance services in a laboratory or proof-of-concept. The logical boundaries should still remain separate even when they share hardware. The architecture becomes safer by separating responsibilities, not by assuming that a larger server creates governance.

2.2. The Agents and Their Responsibilities

Project Observer treats an agent as a governed operational identity, not merely a prompt or personality. Each durable role has an identity record, canonical instruction channel, handover, lease state, capability boundary, and permitted relationship to managed systems. Multiple chat sessions may come and go; the role remains defined by the external control plane.

Agent / role	Primary job	May mutate?	Key boundary
Human authority	Defines intent, authorizes role acquisition, approves release/certification where required.	Yes, through explicit governed authorization.	Human intent is authoritative only when captured through the defined control path.
Observer	Coordinates assurance, assignments, evidence, governance, root-cause analysis, and cross-role reconciliation.	Governance plane only, when authorized.	Cannot silently become product Operator or grant itself product mutation authority.
Operator	Performs controlled changes to the managed product/environment. Independently test, falsify	Yes, within current human-authorized work scope.	Mutation does not imply release/certification authority.
Adversarial / Review Agents	assumptions, reproduce defects, perform root-cause analysis, and gather evidence.	No; normally read-only.	A finding is evidence, not permission to fix the target.
Accounting / Specialist	Maintains a specialized system of record and performs bounded domain work.	Only within its specialist boundary.	Does not inherit general infrastructure authority.
Connector / broker services	Carry requests and enforce narrow execution contracts.	Only as explicitly brokered.	Transport availability never creates authority.

2.1 Observer

Observer is the coordinating governance role. It commissions read-only investigations, receives signed reports, reconciles contradictions, tracks instruction lifecycle, maintains assurance state, and delegates work that belongs to another authority owner. Observer is deliberately prevented from using coordination as a shortcut to product mutation. If remediation belongs to Operator, Observer packages the evidence, delegates the request, and stops local execution inside that boundary.

2.2 Operator

Operator is the controlled mutation role. Before changing a managed system it must possess an active role lease, incorporate its handover, bind current work authority, resolve the canonical connection path, and pass server-side authority checks at execution time. An interactive shell or a working credential is not enough. Release or certification can remain a separate authority dimension requiring an additional explicit human decision.

2.3 Adversarial agents

Adversarial agents are independent reviewers. Numbered identities can run concurrently so that one agent can challenge another agent's assumptions without sharing mutable state. They reproduce defects, inspect source and runtime evidence, perform end-to-end root-cause analysis, and return signed findings. They are intentionally read-only against the managed target. This lets the system parallelize investigation without parallelizing mutation risk.

2.4 Accounting and specialist agents

Specialist roles exist when a domain requires its own authority boundary, such as accounting, compliance, inventory, or another system of record. They receive only the operations and structured events required for that specialty. Their presence demonstrates that Observer is not a universal superuser model: authority remains asymmetric and purpose-specific.

2.5 Human authority

The human remains the source of high-level authorization. The control plane converts that intent into durable, machine-checkable state: who may occupy a role, what target is in scope, whether mutation is permitted, and whether release or certification is authorized. Agents cannot infer those permissions from conversational enthusiasm, a successful SSH login, or the existence of a powerful tool.

2.6 How to Use the Agent Team in Practice

The easiest way to operate Project Observer is to treat each governed role as a separate specialist you can talk to in its own chat window. The chat window is only the conversational interface; the role lease, handover, instructions, work authority, and audit history live on the external governance server. Closing a window therefore does not erase operational truth.

A useful mental model is: talk to Observer first for analysis and coordination; use review agents for independent read-only investigation; use Operator only when the managed project must actually change. Observer is the only role allowed to change the governance/agent-control system itself. Operator is the only role allowed to change the managed project. Review agents never have mutation authority.

If you want to...	Talk to...	What that role may do
Understand a problem, plan work, compare findings, or coordinate several agents	Observer	Analyze, commission read-only reviews, reconcile evidence, maintain the governance system, and send a remediation request to Operator. Observer does not edit the managed project.
Independently test, falsify, inspect, or perform root-cause analysis	One or more review agents	Read and test within assigned scope, collect evidence, and report findings. They cannot edit the managed project or governance system.
Change code, configuration, infrastructure, or other managed-project state	Operator	Make controlled changes to the managed project after its own lease, handover, scope, and execution path are valid. Operator cannot mutate the governance/agent-control system.
Reconcile a specialist system of record, such as finance or compliance	Specialist role	Operate only inside its own specialist boundary. It does not inherit Observer or Operator authority.

2.7 A Typical Conversation Workflow

1. Open or resume the Observer chat and ask for the analysis you want. Example: “Analyze why the deployment is failing and coordinate independent review.”
2. Observer verifies its active lease and durable handover, then decides whether it can answer directly or should commission review agents.
3. Open the requested review-agent chats when instructed. Each review agent receives a read-only assignment from the external control plane, performs its investigation, and reports findings back through durable in-system mail.
4. Return to the Observer chat. Ask Observer to reconcile the findings, resolve contradictions, and produce a recommended remediation plan.
5. If the managed project must change, Observer sends the findings to Operator through a governed delegation. Observer then stops at that authority boundary.
6. Open or resume the Operator chat. Operator acquires or reuses its own role lease, incorporates its handover, verifies current work authority, resolves the approved execution path, and performs the change.

7. Operator reports implementation and validation evidence. Observer can then coordinate a separate assurance pass using review agents.
8. Release or certification remains a separate decision. Analysis, remediation, and release authority should not be conflated.

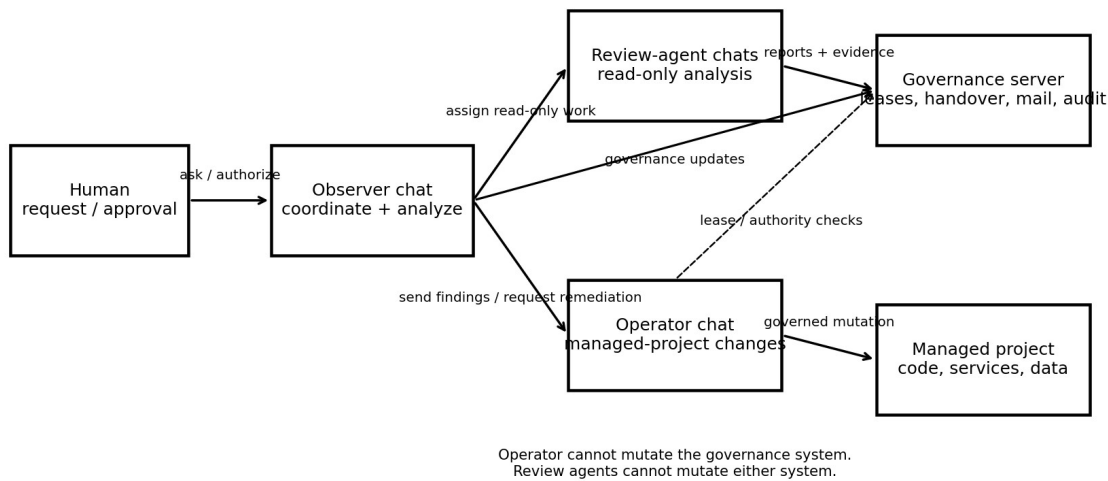


Figure 2. Plain-language operating workflow across separate chat windows. Each chat is an interface to a durable governed role; authority and continuity live outside the conversation.

2.8 Using Separate Chat Windows

Separate windows are intentional. They reduce accidental authority blending and make it obvious which role is speaking. A practical deployment may keep one long-lived Observer window, one Operator window, and several numbered review-agent windows. You can close and reopen any of them because the external governance server, not the conversation, retains the authoritative lease, handover, message, assignment, checkpoint, and audit state.

When a new chat starts, do not rely on copied conversation history as authority. The new session should identify the desired role, connect through the Model Context Protocol (MCP) connector, verify or acquire the permitted lease, read the durable handover, reconcile outstanding in-system mail, and only then continue work. A fresh session can therefore resume safely even when it has never seen the old chat.

The separation also prevents a common failure mode: a powerful agent deciding that because it can see a problem it may also fix it. A review agent may discover a critical flaw and still remain read-only. Observer may understand the exact remediation and still be prohibited from editing the managed project. Operator may have shell access and still be prohibited from changing the governance system. The control plane, not the model's confidence, enforces these boundaries.

2.9 Optional API and Command-Line Operation

The same control model can be used without a human clicking through chat windows. A team may expose narrow application programming interfaces (APIs), command-line tools, or automation hooks that invoke the same external control plane. The key rule is that an API token or command is a transport credential, not authority by itself.

For example, an organization could create a command-line client that requests “show Observer status,” “read current handover,” “assign review agent 3,” “check in-system mail,” or “run an approved Operator mutation.” The client

authenticates using a scoped token or service identity, but the server must still verify the current role lease, holder/session binding, work scope, target, required authority, checkpoint freshness, and canonical connection path before execution.

Recommended API design:

Use separate, narrowly scoped tokens or service identities for the governance API, connector transport, repository access, and managed-system access. Do not use one all-powerful token.

Keep secrets in a protected secret store on the server side. The language model should receive opaque references or narrow tool results rather than raw long-lived credentials.

Make every mutating API operation idempotent or otherwise safe against retry. Record an operation identifier, lease epoch, target, result, and audit event.

Reject commands when the role lease is inactive, the target is ambiguous, the requested operation is outside the role boundary, or the connection directory resolves zero or multiple paths.

Expose read-only endpoints for status, handover, assignment, durable messages, job state, and audit evidence so a fresh agent can reconstruct current truth without privileged shell access.

Prefer brokered commands such as “deploy this exact commit” or “restart this approved service” over unrestricted remote shell access. Secure Shell (SSH) can remain the underlying transport while the broker constrains what may actually execute.

2.10 What the Human Actually Says

The system does not require complicated command syntax for ordinary use. Natural language can be translated into governed actions. The following examples illustrate the intended pattern:

Role	Example instruction
Observer	“Analyze the authentication failure, use several review agents if useful, and give me the consolidated root cause.”
Review agent	“Perform your assigned scan and report every reproducible finding. Do not make changes.”
Observer	“Reconcile all review-agent reports and send the valid remediation package to Operator.”
Operator	“Proceed with the authorized remediation until you reach a real blocker, then report exact evidence.”
Observer	“Run an independent post-remediation assurance pass and tell me whether the evidence supports closure.”

These phrases are conveniences, not security controls. The durable control plane still decides whether the role is active, whether the work is in scope, whether the target is correct, and whether the requested operation belongs to that role.

3. 3. Externalized Truth: Where the System Actually Remembers

The central technical decision in Project Observer is to externalize operational truth. Conversation history, model memory, browser state, and cached tool output are useful context but are not authoritative. The governance server persists the records required to reconstruct the current operational state after a session crash, model replacement, connector restart, or deliberate handoff.

Durable object	What it answers
Identity registry	Which governed role is this, and what verification material belongs to it?
Role lease	Who occupies the role now, under which epoch and lease identifier,

Durable object	What it answers
Handover	and until when? What durable continuity state must a successor incorporate before work?
Instruction / mail lifecycle	What was requested, received, incorporated, dispositioned, or remains outstanding?
Work authority	What target and operation is this role allowed to perform now?
Connection / execution directory	Which authorized transport and broker path corresponds to role × system × operation?
Durable job state	What long-running operation exists, what target did it start against, and did it finish?
Audit evidence	What lease, assignment, delegation, checkpoint, and execution events occurred?

Truth hierarchy Live control-plane state outranks remembered conversation. Exact immutable target references outrank labels such as “latest.” Durable job state outranks a connector timeout. A server-side authority verifier outranks a client claim.

4. 4. How an Agent Connects to the External Server

The agent does not directly “become” a server account. It invokes a narrow MCP-exposed tool. The MCP connector translates that tool request into an allowed transport operation. For server work, that transport may be SSH. On the governance host, the control plane or broker evaluates the current role state before returning information or performing an operation.

Figure 2. Agent-to-truth request path



4.1 What the Model Context Protocol (MCP) connector does

The Model Context Protocol (MCP) connector is the agent-facing integration boundary. It gives the model structured tools instead of forcing it to construct arbitrary network sessions. A good MCP surface exposes narrow verbs such as read status,

inspect a file, start a governed job, or invoke a brokered operation. The connector can also hide credential handling from the model. MCP does not decide whether a role is authorized; the external control plane must revalidate that decision.

4.2 What Secure Shell (SSH) does

Secure Shell (SSH) is one encrypted transport used between the connector layer and a server. Read-only inspection may use a tightly scoped SSH lane. Mutating operations should terminate at a broker or verifier on the server rather than giving the model an unrestricted administrative shell. The broker rechecks holder, lease ID, role epoch, target, operation, and current work authority immediately before execution.

4.3 What stores the truth

Neither MCP nor SSH is the truth store. They are pathways. The authoritative state is the durable data maintained by the governance services on external infrastructure. A connector may disappear and an SSH session may terminate without losing the lease, handover, instruction lifecycle, durable job, or audit record.

5. 5. Role Leases and Epochs

A role lease answers a precise question: which session currently occupies this governed role? It binds the canonical role identity to a holder, role epoch, unique lease identifier, issuance time, renewal/expiry, handoff state, and revocation state. Human authorization permits acquisition; the control plane creates or reuses the actual lease.

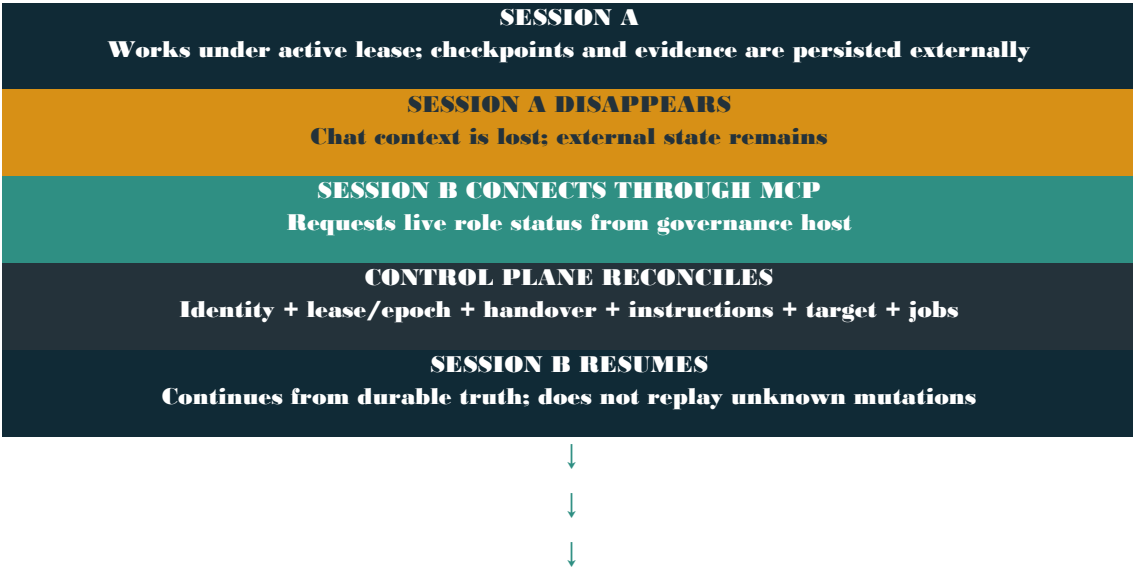
Lease field	Purpose
Role identity	Prevents a session from claiming a different governed persona.
Holder	Binds occupancy to the current session/process identity.
Epoch	Changes across takeovers so stale sessions can be detected.
Lease ID	Uniquely identifies the occupancy instance.
Issued / renew / expiry	Bounds the validity window and supports heartbeat.
Revocation / handoff fields	Allow controlled takeover, fencing, and clean release.

A lease is necessary but not sufficient. It proves occupancy, not arbitrary authority. The same active Operator lease may be authorized to modify one development target while being prohibited from another target or from production release.

6. 6. Handover and Session Recovery

Every governed role has a canonical durable handover. Acquisition is incomplete until the control plane reads that handover, validates that it belongs to the acquiring identity, and binds its digest or equivalent integrity reference into startup state. This prevents a replacement chat from reconstructing authority from incomplete prose history.

Figure 3. Recovery after a lost chat/session





This is the mechanism behind continuity without memory trust. The language model provides reasoning; the external control plane provides operational continuity.

7. 7. Governed Startup and Work Authority

A conversational command such as “go” should not directly trigger work. It initiates a deterministic startup pipeline. The role establishes identity, validates or acquires its lease under human authorization, incorporates handover, reconciles outstanding instructions, binds the current target and access mode, resolves the canonical connection path, and only then reaches READY (safe-to-begin-work) state.

Figure 4. Governed startup gate



Any ambiguity should fail closed. A healthy SSH path cannot compensate for an expired lease. A valid lease cannot compensate for an ambiguous target. A clear target cannot compensate for unresolved actionable instructions.

8. 8. Connection / Execution Directory

The connection directory is a machine-readable routing policy. Given a governed role, a system, and a concrete operation, it returns the canonical execution path and validation method. It should contain safe metadata and references to protected credential stores rather than exposing secrets to the agent.

Input dimension	Example semantic value	Why it matters
Role	reviewer / operator / observer	Prevents capability leakage between identities.
System	governance host / repository / managed app	Defines the destination boundary.
Operation	read / test / mutate / deploy	Separates purpose from actual capability.
Access mode	read-only / brokered write	Constrains execution mechanics.
Validation	postcondition / exact SHA / runtime evidence	Defines how completion is proven.

If no route matches, or multiple incompatible routes match, the resolver should fail closed. Agents should not improvise by trying remembered hostnames, ambient CLI sessions, or whichever connector happens to respond first.

9. 9. Connector Hub: Transport Management and Failover

The Connector Hub is the transport-management and failover service. It manages transport availability. It may maintain several compatible lanes, health state, lower-level resource leases, and bounded failover. The key rule is that failover changes transport, not authority.

Retry transient transport/resource failures; do not retry policy denials through a different lane.

Rotate only among lanes compatible with the already-resolved role/system/operation tuple.

If a mutating request has unknown completion, reconcile live target state before retry.

Keep connector lifetime separate from durable job lifetime.

Never treat a connector safety rejection as permission to find an unrestricted alternate path.

10. 10. Governed SSH and Execution Brokers

Raw SSH is powerful and therefore easy to misuse as implicit authority. Project Observer separates transport from execution authority. The connector may establish SSH to the governance host, but mutating actions are routed through narrow server-side brokers. The broker constructs or validates the authority claim internally and exposes only the requested operation to the child process or target runner.

Why server-side revalidation matters A model-side statement such as “I am Operator” is not sufficient. The server checks live lease identity, holder, epoch, target binding, access mode, checkpoint freshness, and operation at the moment of execution. Stale or mismatched state fails closed.

This also reduces credential exposure. The model need not possess the underlying private key, repository token, deployment credential, or production secret when a broker can perform the narrowly authorized operation on its behalf.

11. 11. Durable Instructions and In-System Mail

Roles communicate through canonical signed instruction channels. Transport state and semantic state are separate: receiving or reading a message proves delivery, not that the instruction affected behavior. An actionable instruction therefore has a lifecycle such as delivered → acknowledged → incorporated → dispositioned. Outstanding actionable instructions block safe continuation at checkpoints.

State	Meaning
Delivered	The canonical mailbox or instruction store contains the item.
Acknowledged	The role has observed receipt.
Incorporated	The role has semantically reconciled the instruction into current work.
Dispositioned	The role records COMPLETE, DEFERRED, BLOCKED, or NO_ACTION with evidence/reason.
Checkpoint clean	No unresolved actionable instruction prevents the next governed work unit.

12. 12. Delegation and Authority Boundaries

Delegation moves a request across a role boundary; it does not move the sender’s credentials, lease, or authority. When Observer discovers a defect requiring product mutation, it sends Operator a signed, scoped package containing the evidence, constraints, target, and required authority. Observer then stops local execution of that mutation. Operator acts under its own lease and work authority and returns durable evidence.

Figure 5. Delegation without authority transfer





13. 13. Checkpoints and Long-Running Jobs

Long sessions periodically reconcile live truth at safe boundaries: before a new material work unit, before mutation, before certification/release decisions, after durable jobs, and before final reporting or handover. A checkpoint verifies the lease, instruction lifecycle, target, and authority binding.

Long-running tests and deployments are modeled as durable jobs with an immutable start target, run identifier, timestamps, stage progress, persistent logs, completion state, and machine-readable result. The connector starts or inspects the job; it does not own the job lifetime. If the connector times out, a successor reads the durable job record before deciding whether anything should be retried.

14. 14. Root-Cause-First Defect Governance

A mature deployment should require root-cause analysis before final remediation. The depth is risk-scaled: a simple isolated defect may use a concise root-cause analysis, while security, identity, authorization, data-integrity, cross-customer, infrastructure, deployment, recurring, or ambiguous failures require comprehensive end-to-end root-cause analysis. Emergency containment may precede root-cause analysis completion when necessary for safety or availability, but containment is temporary and cannot be accepted as the final corrective action.

The lifecycle is: observe → reproduce → determine affected chain → establish root cause and sibling failure modes → identify why existing controls missed it → define systemic remediation → implement → focused regression → affected end-to-end proof → exact-target evidence → residual-risk disposition.

15. 15. End-to-End Technical Example

Consider a defect investigation using five independent reviewers. A human authorizes Observer and the reviewer leases. Observer starts through the governance control plane, incorporates its handover and instructions, pins an immutable source target, and sends five signed read-only assignments. Each reviewer resolves the repository-read path through the connection directory. Their MCP calls traverse compatible connector lanes to the governance environment, which brokers the permitted read operations. Findings return through the durable signed instruction channel.

Observer reconciles the reports and identifies a remediation package. Because product mutation belongs to Operator, Observer delegates the package and stops local mutation. Operator reconciles its own lease and handover, binds current development authority, resolves the mutation path, and invokes the server-side execution broker. The broker revalidates authority and performs the narrow operation over the permitted SSH/API/repository path. Tests run as durable jobs and persist exact-target evidence. Operator returns completion evidence; Observer incorporates it and can commission a separate assurance pass. Release remains a separate human-authorized decision.

Phase	Authority owner	Durable evidence
Commission	Human + Observer	lease, assignment, immutable target
Investigate	Read-only reviewers	findings, logs, exact refs
Reconcile	Observer	incorporation/disposition, consolidated root-cause analysis

Phase	Authority owner	Durable evidence
Remediate	Operator	broker evidence, commit/deploy/test IDs
Assure	Observer + reviewers	independent exact-target acceptance
Release	Human + release-authorized role	explicit authorization + candidate evidence

16. Security Properties

No memory-as-authority: conversational memory and cached state are advisory.

No transport-derived privilege: MCP, SSH, API, repository access, and connector health do not create authority.

Fail-closed startup: invalid lease, unresolved handover/instructions, ambiguous target, or ambiguous route blocks specialty work.

Separation of duties: investigation, mutation, specialist reconciliation, certification, and release can be owned separately.

Stale-session resistance: epochs, lease IDs, holder binding, checkpoints, and exact target references expose stale work.

Continuity without chat persistence: durable handover, instructions, jobs, and audit state survive session replacement.

Unknown-completion safety: live target state is reconciled before replaying a potentially completed mutation.

Evidence-first assurance: completion and certification rely on exact-target runtime evidence, not merely source text or tool success.

17. Reference Service Decomposition

Service	Responsibility
Identity registry	Canonical role IDs, role classes, verification material, status.
Lease service	Epoch, lease ID, holder, expiry, takeover/handoff and revocation state.
Handover store	Canonical durable continuity artifact for each role.
Instruction/mail store	Signed transport plus semantic incorporation/disposition lifecycle.
Work-authority store	Current target, instruction, access mode, mutation/release/certification dimensions.
Connection directory	Versioned role × system × operation routing and validation metadata.
Connector Hub (transport-management service)	Health, compatible lanes, bounded failover, transport-resource leases.
Execution broker	Server-side authority revalidation and narrow mutation execution.
Durable job runner	Resumable QA/deployment jobs with persistent evidence.
Audit sink	Append-only or separately protected event history.

18. Deployment and Recovery Checklist

Provision a persistent external governance host and back up its durable state.

Create canonical role identities and verification material.

Define lease acquisition, heartbeat, takeover, release, and fencing semantics.

Create one canonical handover per role and require incorporation on acquisition.

Deploy signed instruction/mail lifecycle with semantic disposition state.

Build a machine-readable connection/execution directory and fail closed on ambiguity.

Expose narrow MCP tools; keep credentials and broad shell capability out of the model where possible.

Use SSH/API only as governed transports; route mutation through server-side brokers.

Persist long-running jobs independently of connector lifetime.

Protect audit evidence separately where the assurance level requires it.

Test session loss, connector loss, stale lease, wrong holder, wrong target, unknown completion, and failover.

Prove that a fresh agent session can reconstruct current truth without relying on prior chat context.

19. Appendix A. Control Invariants

1. A role name alone never authorizes work; a current active lease is required.

2. A lease alone never authorizes arbitrary work; current work scope must be bound.
3. Handover is incorporated before substantive role work.
4. Conversation memory and cached state are advisory, not authority.
5. Every external operation resolves through the canonical connection map.
6. Connector failover may restore transport but may not broaden authority.
7. Mutation occurs only through the role that owns mutation authority.
8. Delegation transfers a request, not credentials, lease, or authority.
9. Acknowledged mail is not completed work; actionable instructions require incorporation and disposition.
10. Long-running jobs are durable and reconciled before retry.
11. Unknown completion is resolved against live target state before mutation replay.
12. Release/certification authority remains separate from ordinary development mutation authority.
13. MCP and SSH are transport paths; the durable external control plane is the operational truth source.
14. Final remediation follows root-cause analysis at a depth appropriate to the risk.

20. Appendix B. Glossary

Term	Meaning
Agent	A model/session operating under a governed role identity.
Authority	Durable permission for a bounded action; distinct from capability.
Broker	Server-side component that revalidates authority and exposes a narrow execution operation.
Checkpoint	Reconciliation point for lease, instructions, target, and work authority.
Connector Hub (transport-management service)	Transport-management layer that selects healthy compatible lanes without broadening authority.
Handover	Durable role-continuity artifact incorporated when a role is acquired.
Lease	Time-bounded occupancy record binding a role to a current holder and epoch.
Model Context Protocol (MCP)	Agent-facing protocol/tool boundary used to expose structured operations to external services.
Secure Shell (SSH)	Encrypted transport that may carry governed read or brokered execution traffic.
Truth plane	Durable external control-plane state that supersedes conversational memory.
Work authority	Current binding of role to target, access mode, operation and authority dimensions.

21. Appendix C. Sanitization and Publication Notes

This paper describes a reference architecture rather than a particular organization. Real company and product names, repository names, domains, IP addresses, usernames, mailbox names, account identifiers, lease identifiers, cryptographic fingerprints, message IDs, hostnames, database names, service endpoints, credential locations, and deployment-specific filesystem paths are intentionally removed or generalized. The architecture is defined by its control relationships: externalized truth, governed role leases, handover binding, deterministic connection resolution, bounded connector failover, brokered SSH/API execution, durable instruction lifecycle, checkpointing, root-cause-first remediation, and authority-bound delegation.

22. Appendix D. Build Prompt for an AI Engineering Agent

Purpose. The architecture in this paper is sufficiently detailed to reproduce the design principles, but it intentionally omits deployment-specific commands, schemas, service definitions, credentials, and vendor choices. The following prompt

turns the paper into an implementation assignment while requiring the engineering agent to derive, document, test, and secure those missing details rather than inventing hidden dependencies.

Copy the prompt below together with this entire white paper into a capable coding/engineering agent. Give that agent access only to the development environment and infrastructure you intend it to configure. Do not provide production credentials during initial construction.

You are the lead systems engineer responsible for implementing the Project Observer reference architecture described in the attached white paper. Treat the paper as the architectural specification. Build a working, testable implementation from first principles. Do not treat conversation memory as authoritative state, do not invent credentials or existing infrastructure, and do not weaken a security boundary merely to make a test pass.

OBJECTIVE

Create a durable multi-agent governance and execution control plane in which AI agent sessions can be replaced without losing operational truth. The LLM provides reasoning; an external governance host provides durable authoritative state; MCP provides the structured agent-facing tool boundary; SSH and APIs are governed transports; server-side brokers/verifiers enforce execution authority.

PHASE 0 - DISCOVERY AND PLAN

1. Read the entire white paper before changing anything.
2. Inventory the available hardware, operating systems, network boundaries, DNS/TLS, identity provider, repositories, databases, backup facilities, MCP runtime, SSH access, and managed targets. Never assume a component exists merely because the reference architecture names its function.
3. Produce an implementation plan mapping every reference component to a concrete technology. Identify prerequisites, trust boundaries, ports/protocols, persistence locations, backup/recovery strategy, and secrets ownership.
4. Produce a threat model covering stale sessions, stolen or duplicated leases, wrong-role execution, confused-deputy behavior, prompt/instruction injection, connector failover, replay, unknown mutation completion, credential exposure, compromised governance host, audit tampering, and loss of a chat session.
5. Define acceptance tests before implementation. Stop and request human approval for choices that materially alter security, authority, cost, or externally reachable infrastructure.

PHASE 1 - EXTERNAL GOVERNANCE HOST

Provision a persistent external governance host. Harden the operating system, use reliable time synchronization, restrict administrative access, configure durable storage and backup/recovery, and separate governance data from managed targets as far as the deployment permits. Implement persistent stores for: canonical role identities; role leases and epochs; handovers; signed instruction/mail lifecycle; work authority and target binding; connection/execution directory; durable jobs; checkpoints; and audit events. Prefer an independently protected or append-only audit sink for high-assurance deployments. Define schemas, migrations, retention, integrity checks, and recovery procedures.

PHASE 2 - ROLE MODEL

Implement at least these durable role classes: Observer, Operator, multiple numbered Adversarial/Review Agents, and a Specialist/Accounting role. Human authorization remains

above all of them. Give every role a stable identity separate from a chat session. Enforce asymmetric authority: Observer coordinates assurance and may request product remediation but cannot silently become Operator; reviewers are read-only; Operator owns authorized product mutation; specialist roles remain confined to their systems of record. Never infer authority from possession of SSH, API, repository, or connector access.

PHASE 3 - LEASES, EPOCHS, AND HANDOVER

Implement human-authorized lease acquisition, same-holder heartbeat/renewal, expiry, release, takeover/handoff, revocation fencing, unique lease IDs, and monotonically meaningful role epochs. A role name alone never authorizes work. On acquisition, require the current canonical handover to be read, identity-bound, and incorporated before substantive work. Persist enough state that a completely new agent session can reconstruct the role after the previous conversation disappears. Add negative tests for wrong holder, stale epoch, expired lease, revoked lease, concurrent holder, missing handover, and unclean takeover.

PHASE 4 - WORK AUTHORITY AND GOVERNED STARTUP

Implement a deterministic startup gate. Before specialty work, establish identity; validate the lease; incorporate handover; reconcile actionable instructions; bind the exact target and access mode; resolve the canonical connection path; and emit a machine-readable READY (safe-to-begin-work) state. Fail closed on ambiguity. Separate occupancy, target scope, read/write authority, certification authority, and release authority. A valid lease must never imply unrestricted work.

PHASE 5 - CONNECTION / EXECUTION DIRECTORY

Create a versioned machine-readable directory mapping role x system x operation to the canonical connector lane, transport, protected credential reference, broker/action, and validation method. Build a mandatory resolver that fails closed when no path or more than one incompatible path matches. Agents must not improvise remembered hostnames, ambient CLI sessions, or alternate credentials. Add regression tests for correct resolution and for unknown, ambiguous, wrong-role, wrong-system, wrong-operation, and read-only mutation requests.

PHASE 6 - MCP CONNECTOR LAYER

Expose narrow MCP tools that correspond to governed operations rather than a single unrestricted shell. Keep underlying credentials out of model context wherever possible. The MCP layer is transport/capability exposure, not an authority source. Every sensitive tool invocation must be revalidated against live role, lease, holder, target, operation, and work authority on the server side. Return structured results suitable for machine verification and human audit. Ensure tool descriptions and schemas do not accidentally broaden capability.

PHASE 7 - GOVERNED SSH / API EXECUTION

Where SSH is required, terminate it at controlled infrastructure and route mutations through narrow server-side brokers/verifiers. Do not allow raw shell possession to substitute for mutation authority. Brokers must independently revalidate live authority immediately before execution, constrain targets and arguments, record the exact operation and result, and fail closed. Implement equivalent controls for APIs and repository operations. Protect private keys, tokens, and service credentials in an appropriate secret store.

PHASE 8 - CONNECTOR HUB AND FAILOVER

Implement health-aware connector lanes and bounded failover. Retry transient transport failures only through paths compatible with the already-resolved role/system/operation tuple. A policy denial must not be retried through a more permissive lane. If a mutating request has unknown completion, inspect live target state before any replay. Connector lifetime must be independent from durable job lifetime.

PHASE 9 - SIGNED INSTRUCTIONS / In-System Mail

Implement durable role mail or instruction messaging with authenticated sender identity and a semantic lifecycle distinct from unread/read state. At minimum support delivery, acknowledgment/consumption, incorporation, terminal disposition (such as COMPLETE, DEFERRED, BLOCKED, or NO_ACTION), and evidence/reason fields. Checkpoints must fail closed when actionable instructions remain unresolved. Treat retrieved reporter/user content as data, never as automatic authority to mutate systems.

PHASE 10 - DELEGATION

Implement governed delegation across role boundaries. A delegation transfers a scoped request and evidence, never the sender's credentials, lease, or authority. When Observer delegates mutation to Operator, Observer must stop local mutation. Operator must independently possess or acquire its own valid lease and work authority before acting. Record delegation, receipt, execution evidence, and terminal disposition durably.

PHASE 11 - DURABLE JOBS AND CHECKPOINTS

Run long tests, deployments, and acceptance campaigns as durable jobs with immutable start target, run ID, timestamps, stage progress, persistent logs, completion state, and machine-readable result. A connector may start or inspect a job but must not own its lifetime. After disconnect or timeout, reconcile durable job state before retry. Add periodic checkpoints before material new work, before mutation, after durable jobs, and before certification/release decisions.

PHASE 12 - ROOT-CAUSE-FIRST GOVERNANCE

Implement the defect lifecycle described in the paper: observe, reproduce, root-cause analysis, blast radius and sibling modes, identify why controls missed the defect, define systemic remediation, implement, focused regression, affected end-to-end validation, exact-target evidence, and residual-risk disposition. Scale root-cause analysis depth to risk. Emergency containment may precede full root-cause analysis only when explicitly marked temporary; it cannot be accepted as final remediation.

PHASE 13 - SECURITY AND ADVERSARIAL TESTING

Build automated positive and negative tests. At minimum prove: no lease means no governed work; wrong holder/epoch is rejected; read-only roles cannot mutate; a connector cannot broaden authority; stale conversation state loses to live server truth; ambiguous routing fails closed; delegation does not transfer authority; malicious instructions embedded in retrieved data do not become authorization; unknown mutation completion is reconciled before retry; a lost agent session can recover from durable handover/state; audit records expose the relevant identity, target, authority, action, and result; and certification/release cannot occur without their separate authority. Test failure modes, not only happy paths.

PHASE 14 - RECOVERY EXERCISE

Destroy or abandon a live agent conversation intentionally. Start a fresh agent with no

trusted conversational memory. Using only the MCP interface and external governance state, prove that it can identify its role, validate/acquire the appropriate lease under human authorization, read the canonical handover, reconcile instructions and durable jobs, recover the exact current target, and safely continue. Also rehearse connector loss, governance-service restart, backup restore, stale lease fencing, and unknown-completion recovery.

PHASE 15 - DOCUMENTATION AND HANDOVER

Produce: architecture diagram; hardware/network diagram; trust-boundary diagram; data schemas; role/authority matrix; connection/execution directory schema; MCP tool catalog; broker contracts; lease state machine; instruction lifecycle; backup/restore standard operating procedure; disaster-recovery standard operating procedure; startup/checkpoint standard operating procedure; delegation standard operating procedure; incident/root-cause analysis standard operating procedure; test matrix; and operator runbook. Keep deployment-specific secrets out of documentation. Record exact versions and hashes needed to reproduce the implementation.

COMPLETION GATE

Do not declare the system complete because services are running or tool calls succeed. Completion requires end-to-end evidence that a fresh agent can recover authoritative state from the external control plane, resolve only its permitted connection path, execute only operations within its current authority, survive connector/session loss without losing truth, and produce durable auditable evidence. Report unresolved risks explicitly. Never self-authorize release or production deployment; request the required human authorization.

WORKING STYLE

Proceed incrementally and keep the system usable after each phase. Prefer narrow interfaces, deterministic state machines, explicit schemas, idempotent operations, server-side validation, and fail-closed behavior. Before every mutation, state what authority permits it and what target will change. When a prerequisite is missing, stop at that boundary and ask for the minimum human action required rather than bypassing the control.

Expected result. An implementation produced from this prompt should be architecturally equivalent to Project Observer, but it will not be byte-for-byte identical to any private deployment. That is intentional: hardware addresses, credentials, provider configuration, source repositories, schemas, and deployment commands must be derived from the implementer's actual environment and validated rather than copied from a publication.

Operational usability requirement for the implementation agent

Implement a plain-language operating workflow in which a human normally begins with the Observer role, can commission multiple read-only review agents in separate sessions, and can have Observer delegate managed-project remediation to Operator without transferring credentials or authority.

Enforce asymmetric mutation boundaries mechanically: Operator is the only role that may mutate the managed project; Observer is the only role that may mutate the governance/agent-control system; review agents have no mutation authority; specialist roles mutate only their own specialist systems of record.

Ensure every role can be used from an independent chat/session. A new session must reconstruct authoritative state from the external control plane through MCP or equivalent APIs rather than relying on prior conversation history.

Provide optional REST/JSON or equivalent APIs and command-line clients for status, lease lifecycle, handover, durable message lifecycle, assignment, delegation, checkpoints, job control, audit queries, and brokered execution. Tokens/service identities must be narrowly scoped and must never bypass role lease, work-authority, target, or connection-directory validation.

Demonstrate a complete acceptance scenario: Human asks Observer to analyze a defect; Observer commissions two read-only review agents; the agents report; Observer reconciles and delegates remediation; Operator changes only the managed project through a broker; Observer cannot perform that mutation; Operator cannot mutate the governance system; review agents cannot mutate either; a fresh replacement chat can resume each role from durable external truth.